

APAI2025 - LAB01

PULP_Embedded_Programming

Authors: Davide Nadalini, Lorenzo Lamberti, Alberto Dequino, Manuele Rusci, Francesco Conti

Contacts: d.nadalini@unibo.it, alberto.dequino@unibo.it

Links: [GitHub Link \(code\)](#)

Summary

1. Subject(s):
 - PULP architecture, vector sum, matrix-vector mul, profiling code execution;
2. Programming Language: C;
3. Lab duration: 3h
4. Objective: Embedded programming & profiling
you will learn the basics of embedded programming, the pulp architecture, basic operations (sum & matmul), and how to profile your code execution (MAC, cycles) !
5. Assignment:
 - 2 tasks
 - Time for delivery: 2 weeks
 - **Submission deadline: Oct 10th 2025**

How to deliver the assignment

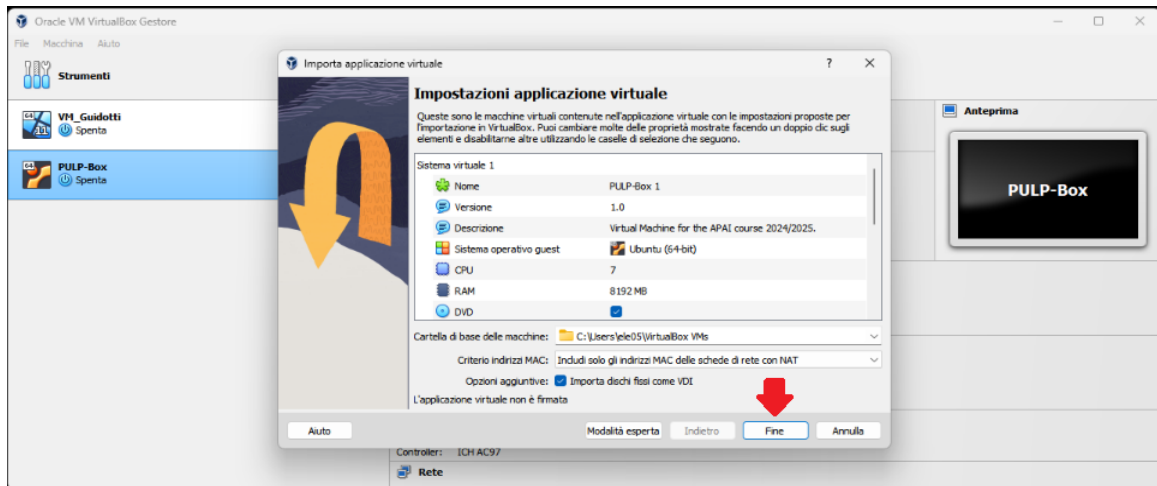
You will deliver ONLY THIS TEXT FILE, no code

- Download this file.
 - Fill in the required results.
 - Export to pdf format.
 - Rename the file to: LAB<number_of_the_lesson>_APAI_<your_name>.pdf
 - Use Virtuale platform to load ONLY your .pdf file
-

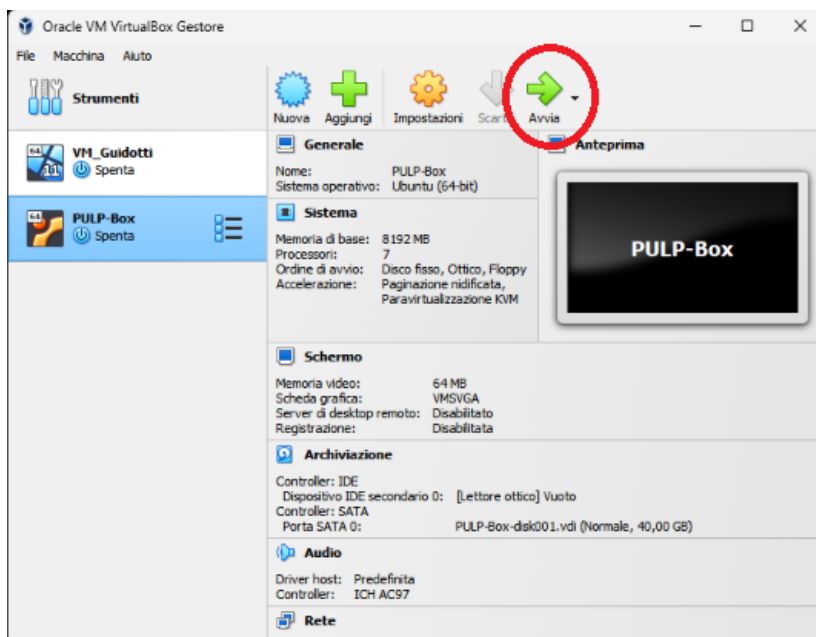
LAB STARTS HERE

0. (ONLY LAB1) Access to the local VM

- On the lab's PCs, open the file explorer and go to This PC, C:/VM_APAI
- Double click on PULP-box.ova
- VirtualBox opens, just click on “Fine”



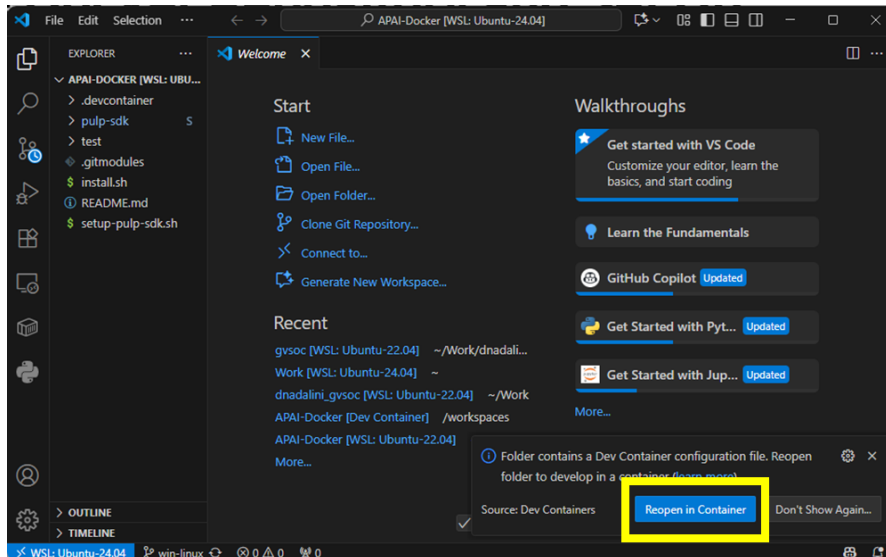
- Wait for the VM to be imported
- Open the VM with “Avvia”



- Password is ‘pulp’

1. (ALL) Open Docker Container with VSCode

- Open a terminal (right click – open a new terminal)
- Go in the APAI-Docker folder: `cd APAI-Docker`
- Open Docker with VSCode: `code .`
- Select the pop-up option “Reopen in container”



- Setup PULP-SDK: `source setup-pulp-sdk.sh`
- Clone GitHub repository of today's lab: `git clone https://github.com/EEESlab/APAI25-LAB01-PULP-Embedded-Programming`
- `cd APAI25-LAB01-PULP-Embedded-Programming`
- `cd pulp-helloworld`
- `make clean all run`

If the output is a “Hello World!”, then everything is correctly set up!

Task 1: vector sum

0. Setup:

- Go to “vector_sum/” folder
- Every time you want to run the code, **SAVE your file** and write in the terminal “**make clean all run**”

1.1. Define N to 50 (N= vector size) and run the code:

	Question	Answer
1	What's the result of the vector_sum() function?	1275
2	Is the checksum correct ?	Yes
3	Print all the elements of "array_1". What's the output?	1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50

Tips:

- **Question 3:** to print the element of the array, uncomment the function call "print_array(array_1, N)" inside the main()

1.2. Define N to 350 (N= vector size) and run the code:

	Question	Answer
1	What's the result of the vector_sum() function?	37105
2	Is the checksum correct ?	No
3	Print all the elements of "array_1". What's the output?	1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 52 53 54 55 56 57 58 59 60 61 62 63 64 65 66 67 68 69 70 71 72 73 74 75 76 77 78 79 80 81 82 83 84 85 86 87 88 89 90 91 92 93 94 95 96 97 98 99 100 101 102 103 104 105 106 107 108 109 110 111 112 113 114 115 116 117 118 119 120 121 122 123 124 125 126 127 128 129 130 131 132 133 134 135 136 137 138 139 140 141 142 143 144 145 146 147 148 149 150 151 152 153 154 155 156 157 158 159 160 161 162 163 164 165 166 167 168 169 170 171 172 173 174 175 176 177 178 179 180 181 182 183 184 185 186 187 188 189 190 191 192 193 194 195 196 197 198 199 200 201 202 203 204 205 206

		207 208 209 210 211 212 213 214 215 216 217 218 219 220 221 222 223 224 225 226 227 228 229 230 231 232 233 234 235 236 237 238 239 240 241 242 243 244 245 246 247 248 249 250 251 252 253 254 255 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 52 53 54 55 56 57 58 59 60 61 62 63 64 65 66 67 68 69 70 71 72 73 74 75 76 77 78 79 80 81 82 83 84 85 86 87 88 89 90 91 92 93 94
4	Array_1 should be filled with increasing values from 1 to N. Why isn't the case here?	Because it is unsigned char and so it has 8 bit for the unsigned number and so can only represent numbers in [0,255], after that limit the result gets truncated (module 256).

Tips:

- **Question 3:** to print the element of the array, uncomment the function call "print_array(array_1, N)" inside the main()
- **Question4:** what range of values a "char" data type can represent?

1.3. Fix the issue: cast "array_1" and the function's arguments to int (or short int)

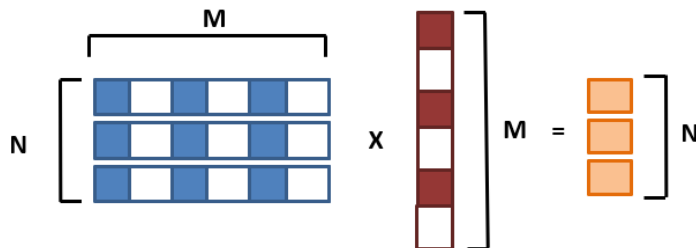
Write down your solution

before	After your fix
<code>unsigned char array_1[N];</code>	<code>unsigned int array_1[N];</code>
<code>int init_array(unsigned char * A_ar, int size)</code>	<code>int init_array(unsigned int * A_ar, int size)</code>
<code>void print_array(unsigned char * A_ar, int size)</code>	<code>void print_array(unsigned int * A_ar, int size)</code>
<code>int vector_sum(unsigned char * A_ar, int size)</code>	<code>int vector_sum(unsigned int * A_ar, int size)</code>

After the fix, answer again

	Question	Answer
1	What's the result of the vector_sum() function?	61425
2	Is the checksum correct ?	Yes
3	Print all the elements of "array_1". What's the output?	1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 52 53 54 55 56 57 58 59 60 61 62 63 64 65 66 67 68 69 70 71 72 73 74 75 76 77 78 79 80 81 82 83 84 85 86 87 88 89 90 91 92 93 94 95 96 97 98 99 100 101 102 103 104 105 106 107 108 109 110 111 112 113 114 115 116 117 118 119 120 121 122 123 124 125 126 127 128 129 130 131 132 133 134 135 136 137 138 139 140 141 142 143 144 145 146 147 148 149 150 151 152 153 154 155 156 157 158 159 160 161 162 163 164 165 166 167 168 169 170 171 172 173 174 175 176 177 178 179 180 181 182 183 184 185 186 187 188 189 190 191 192 193 194 195 196 197 198 199 200 201 202 203 204 205 206 207 208 209 210 211 212 213 214 215 216 217 218 219 220 221 222 223 224 225 226 227 228 229 230 231 232 233 234 235 236 237 238 239 240 241 242 243 244 245 246 247 248 249 250 251 252 253 254 255 256 257 258 259 260 261 262 263 264 265 266 267 268 269 270 271 272 273 274 275 276 277 278 279 280 281 282 283 284 285 286 287 288 289 290 291 292 293 294 295 296 297 298 299 300 301 302 303 304 305 306 307 308 309 310 311 312 313 314 315 316 317 318 319 320 321 322 323 324 325 326 327 328 329 330 331 332 333 334 335 336 337 338 339 340 341 342 343 344 345 346 347 348 349 350

Task 2: matrix-vector product & profiling



0. Setup:

- Go to “matrix_vector_product/” folder
- Every time you want to run the code, **SAVE your file** and write in the terminal “make clean all run”

2.1. Implement missing code

GOAL: We want to count how many Multiply-ACcumulate operations we perform for a GEMV

TASK: Increment a counter every time we perform a MAC inside the GEMV inner loop. I prepared a variable called `mac_counter`. This is the reference code

```
int mac_counter=0;
/* generic matrix-vector multiplication */
int __attribute__((noinline)) gemv(int size_N, int size_M, int *mat_i, int *vec_i, int *vec_o)
{
    for (int i = 0; i < size_N; i++) // outer loop of the gemv
    {
        for (int j = 0; j < size_M; j++) // inner loop of the gemv
        {
            // multiply accumulate operation (MAC)
            vec_o[i] += mat_i[i * size_M + j] * vec_i[j];
            /*(vec_o+i) += *(mat_i+i*M+j)**(vec_i+j); // try to uncomment this and comment the above line
        }
    }
}
```

Put your solution below (code)

```

/* generic matrix-vector multiplication */
int __attribute__((noinline)) gemv(int size_N, int size_M, int *mat_i, int *vec_i, int *vec_o)
{
    /*
    TASK 2.1
    - you must count the MAC by increasing a counter in the inner loop of the gemv.
    - use the "mac_counter" global variable
    */
    mac_counter=0;
    for (int i = 0; i < size_N; i++) // outer loop of the gemv
    {
        for (int j = 0; j < size_M; j++) // inner loop of the gemv
        {
            // multiply accumulate operation (MAC)
            vec_o[i] += mat_i[i * size_M + j] * vec_i[j];

            mac_counter += 1;

            /*(vec_o+i) += *(mat_i+i*M+j)*(*(vec_i+j)); // try to uncomment this and comment the previous line
        }
    }
}

```

2.2. Implement missing code part II

Add performance counters to profile the gemv. The gemv is calculated with this line of code in the main():

```
gemv(N, M, matrix, vector, output_vec);
```

To profile it, exploit these two functions I prepared for you. You should start the profiling right before and stop it right after

```
start_perf_counter();
stop_perf_counter();
```

Put your solution below (code)

```

/* call the GEneric Matrix-Vector (gemv) function */
start_perf_counter();
gemv(N, M, matrix, vector, output_vec);
stop_perf_counter();

```

2.3. Implement missing code part III

Enable the performance counters of our interest. We want to profile:

- Execution cycles (total)
- N° instructions executed

Here's the full list of the performance counters

```
typedef enum {
    PI_PERF_CYCLES      = 17, /*!< Total number of cycles (also includes the
        cycles where the core is sleeping). Be careful that this event is using a
        timer shared within the cluster, so resetting, starting or stopping it on
        one core will impact other cores of the same cluster. */
    PI_PERF_ACTIVE_CYCLES = 0, /*!< Counts the number of cycles the core was
        active (not sleeping). */
    PI_PERF_INSTR       = 1, /*!< Counts the number of instructions executed.
        */
    PI_PERF_LD_STALL    = 2, /*!< Number of load data hazards. */
    PI_PERF_JR_STALL    = 3, /*!< Number of jump register data hazards. */
    PI_PERF_IMISS       = 4, /*!< Cycles waiting for instruction fetches, i.e.
        number of instructions wasted due to non-ideal caching. */
    PI_PERF_LD          = 5, /*!< Number of data memory loads executed.
        Misaligned accesses are counted twice. */
    PI_PERF_ST          = 6, /*!< Number of data memory stores executed.
        Misaligned accesses are counted twice. */
    PI_PERF_JUMP        = 7, /*!< Number of unconditional jumps (j, jal, jr,
        jalr). */
    PI_PERF_BRANCH      = 8, /*!< Number of branches. Counts both taken and
        not taken branches. */
    PI_PERF_BTAKEN      = 9, /*!< Number of taken branches. */
    PI_PERF_RVC         = 10, /*!< Number of compressed instructions
        executed. */
    PI_PERF_LD_EXT      = 12, /*!< Number of memory loads to EXT executed.
        Misaligned accesses are counted twice. Every non-TCDM access is considered
        external (cluster only). */
    PI_PERF_ST_EXT      = 13, /*!< Number of memory stores to EXT executed.
        Misaligned accesses are counted twice. Every non-TCDM access is considered
        external (cluster only). */
    PI_PERF_LD_EXT_CYC  = 14, /*!< Cycles used for memory loads to EXT.
        Every non-TCDM access is considered external (cluster only). */
    PI_PERF_ST_EXT_CYC  = 15, /*!< Cycles used for memory stores to EXT.
        Every non-TCDM access is considered external (cluster only). */
    PI_PERF_TCDM_CONT   = 16, /*!< Cycles wasted due to TCDM/log-interconnect
        contention (cluster only). */
} pi_perf_event_e;
```

Ref: `/rtos/pmsis/pmsis_api/include/pmsis/chips/default.h`

You can enable them in the `start_perf_counter()` function.

Complete the code where you find `/* YOUR CODE HERE */` with the right performance counters (see previous figure)

```
void start_perf_counter()
{
    // enable the perf counter of interest
    pi_perf_conf( 1 << PI_PERF /* YOUR CODE HERE*/ | // count cycles
                 1 << PI_PERF /* YOUR CODE HERE*/ ); // count instructions
    // reset the perf counters
    pi_perf_reset();
    // start the perf counter
    pi_perf_start();
}
```

Put your solution below (code)

```

void start_perf_counter()
{
    // enable the perf counter of interest
    pi_perf_conf( 1 << PI_PERF_CYCLES |    // count cycles
                 1 << PI_PERF_INSTR );    // count instructions
    // reset the perf counters
    pi_perf_reset();
    // start the perf counter
    pi_perf_start();
}

```

2.4. Implement missing code part IV

GOAL: we now are able to measure Cycles & instructions (thanks to performance counters), and MAC operations (your implementation in the code)

TASK: calculate and print the following metrics in the code (fill in where you read `/*YOUR CODE HERE*/`)

- CPI = cycles/instructions
- MAC/cycles
- Instructions/Cycles
- Instructions/MACs

```

/*
TASK 2.2
Measure:
- How many MAC operations are needed for the gemv
Calculate:
- CPI
- MACs/Cycles
- Instructions/Cycles
- Instructions/ MACs
*/

// N° Multiply Accumulate Operations (MACs)
/* already done with task 2.1. use the mac_counter global variable */
// CPI = cycles / n°instructions_executed
float cpi = /* YOUR CODE HERE*/
// MAC/Cycles
float mac_on_cycles = /* YOUR CODE HERE*/
// Instructions/Cycles
float instructions_on_cycles = /* YOUR CODE HERE*/
// Instructions/MAC
float instructions_on_mac = /* YOUR CODE HERE*/

```

Put your solution below (code)

```
// N° Multiply Accumulate Operations (MACs)
| /* already done with task 2.1. use the mac_counter global variable */
// CPI = cycles / n°instructions_executed
float cpi = cycles_counter / (float) instr_counter;
// MAC/Cycles
float mac_on_cycles = mac_counter / (float) cycles_counter;
// Instructions/Cycles
float instructions_on_cycles = instr_counter / (float) cycles_counter;
// Instructions/MAC
float instructions_on_mac = instr_counter / (float) mac_counter;
```

2. The size of the matrix is NxM=50x50 and the vector is M=50. Run the code and fill the table:

Note:

- Profile the matrix-vector product with different compiler optimizations: -O1, -O3, -O3 with HW Loops (default is -O1)

	-O1	-O3 -mnohwlloops	-O3
Cycles	35403	32796	25501
N°Instructions	25396	22793	20496
MACs	2500	2500	2500
CPI	1.394007	1.438863	1.244194
Instructions/Cycles	0.717356	0.694993	0.803733
Instructions/MACs	10.159200	9.117200	8.198400